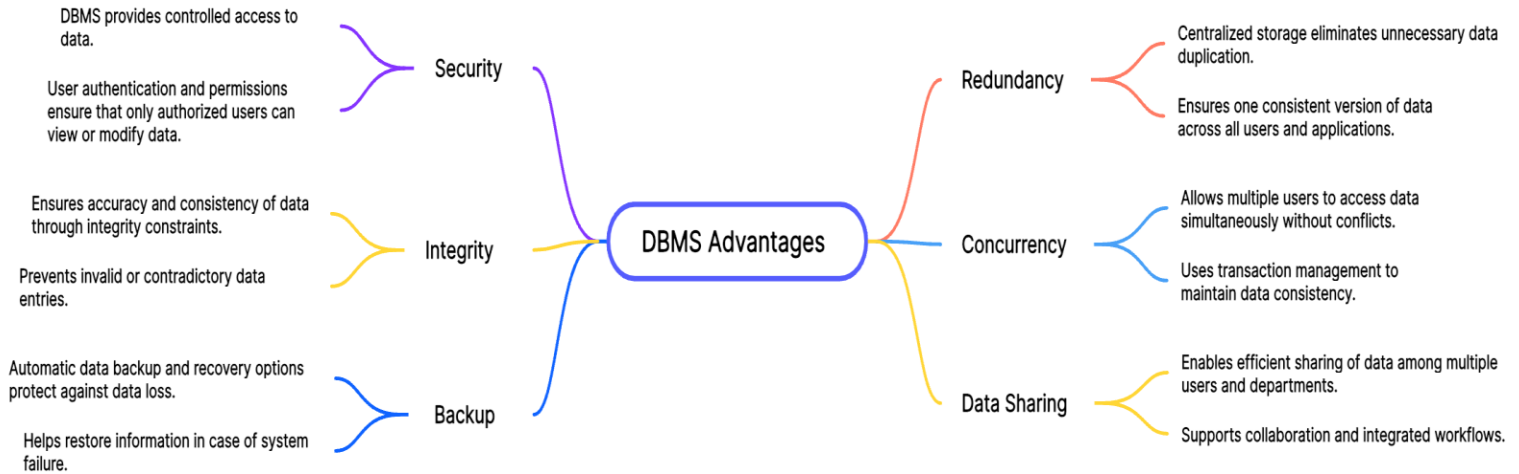


Task Sections & Requirements

1. Comparison Assignment: Flat File Systems vs. Relational Databases

Feature	Flat File Systems 	Relational Databases 
 Structure	 Data is stored in a single file (like .CSV or .TXT).  Each row = record, each column = field.  No relationships between files — all data is independent.  Simple and flat structure without links or constraints.	 Data is organized into multiple related tables.  Each table = one entity (e.g., Students, Courses).  Tables connected using Primary Keys and Foreign Keys.  Supports data linking, reduces redundancy, and improves consistency.
 Data Redundancy	 High redundancy — the same data may appear in many files.	 Low redundancy — data is normalized and referenced by keys.
 Relationships	 No built-in relationships — files are independent.	 Strong relationships between tables via Primary & Foreign Keys .
 Example Usage	 Simple apps like: contact lists, logs, or configuration files.  Used in small datasets or spreadsheets (e.g., Excel, CSV).	 Complex systems like: banking, hospitals, schools, and e-commerce platforms.
 Drawbacks	 Hard to maintain or keep data consistent.  Poor performance with large data.  No support for complex queries or relationships.  High data duplication.	 Requires setup and SQL knowledge.  Needs database software.  Slightly slower for very small datasets due to overhead.

2. DBMS Advantages – Mind Map



🔧 Roles in a Database System

1. System Analyst 🧑

- Studies organizational requirements and current systems.
- Defines functional and technical requirements for the database.
- Acts as a bridge between management and technical teams.
- Important: They often produce requirements documents and use case diagrams to guide the project.

2. Database Designer 🧠

- Designs the database structure: tables, relationships, and keys.
- Performs normalization to reduce redundancy and improve efficiency.
- Plans data organization, storage, and security.
- Important: Their work ensures data integrity and scalability, laying the foundation for efficient database use.

3. Database Developer 💻

- Writes SQL queries, stored procedures, and triggers.

- Develops applications that interact directly with the database.
- Optimizes database performance and implements the design.
- Important: They may also handle data migration and integration with other systems.

4. Database Administrator (DBA)

- Manages and maintains the database system.
- Responsible for backup, recovery, security, monitoring, and performance tuning.
- Ensures the database is available, reliable, and consistent at all times.
- Important: DBAs monitor storage usage, concurrency issues, and ensure compliance with data regulations.

5. Application Developer

- Builds software applications or interfaces that access the database.
- Focuses on user interface (UI) and user experience (UX).
- Connects applications with the database efficiently.
- Important: They ensure smooth data flow and that apps meet business requirements.

6. BI (Business Intelligence) Developer

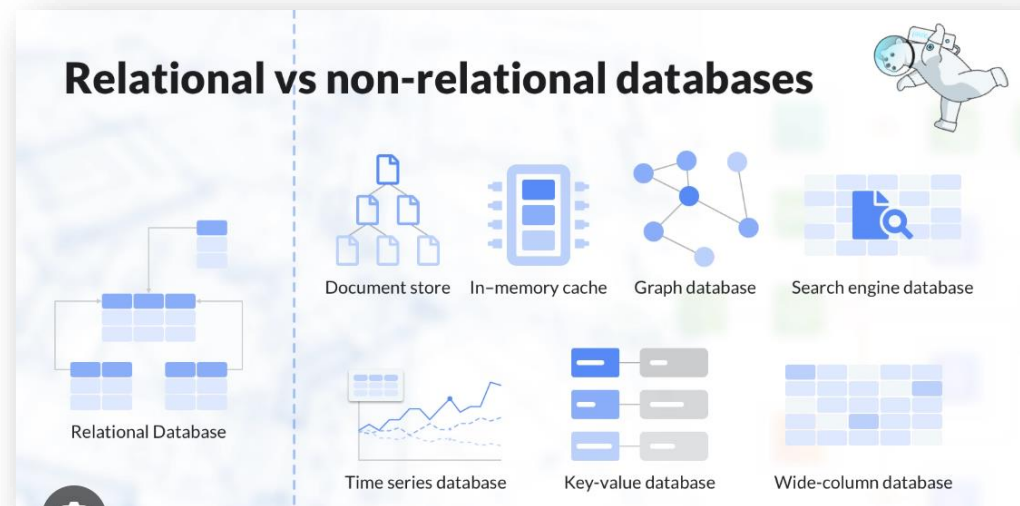
- Transforms raw data into actionable insights for decision-making.
- Creates reports, dashboards, and visualizations.
- Uses analytics tools to identify trends, patterns, and opportunities.
- Important: They often work closely with executives and managers, helping strategic planning and predictive analysis.

4. Types of Databases

Relational vs. Non-Relational Databases:

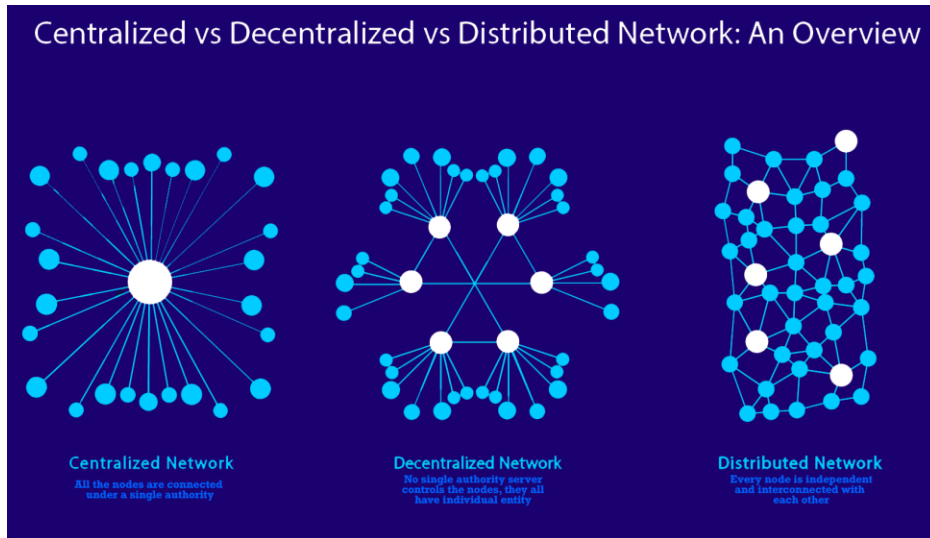
Feature	Relational DB (SQL) 	Non-Relational DB (NoSQL) 
 About it	Stores data in tables with rows and columns. Each table represents one entity (e.g., Employees, Products).	Stores structured, semi-structured, or unstructured data in flexible formats like documents, key-value pairs, graphs, or wide columns.
 Data Model	 Tabular format — each row = record, each column = field.	 Flexible models — documents, key-value, graph, or column-based (e.g., MongoDB uses JSON-like documents).
 Schema	 Fixed schema — structure must be defined before inserting data.	 Dynamic schema — fields can be added anytime without breaking structure.
 Data Types	 Best for structured data (bank transactions, employee info).	 Best for unstructured/semi-structured data (social media posts, user profiles).
 Integrity	 Ensures strong consistency using ACID principles.	 Prioritizes speed and availability using BASE principles.
 Scalability	 Vertical scaling — upgrade one server (CPU, RAM).	 Horizontal scaling — add more servers to handle load.
 Querying	 Uses SQL for querying and managing data.	 Uses custom query languages or APIs (e.g., MongoDB syntax).
 Complexity	 Excellent for complex joins and queries across tables.	 Simple for basic operations, but complex joins are difficult.
 Performance	 May slow down with large datasets and many joins.	 High performance for large-scale data and fast reads/writes.
 Pros	 Secure and reliable (ACID).  Strong table relationships.  Easy navigation and data integration.  Many skilled developers available.	 Highly flexible data model.  Scales easily for big data.  Fast performance.  Adapts quickly to new requirements.
 Cons	 Fixed schema.  Hard to adapt to changes.  Scaling can be expensive.	 Lower consistency.  No standard query language.  Harder to maintain data integrity.
 Use Cases	 Finance, healthcare, e-commerce — need security and reliability .	 Social media, IoT, marketplaces — handle massive unstructured data .

	📁 Stable structure with clear schema. 💰 Budget-friendly (many SQL developers).	⚡ Quick setup for MVPs or fast launches. ☁ Best for speed and scalability.
🧰 Examples	MySQL, PostgreSQL, Microsoft SQL Server, SQLite	MongoDB, Cassandra, Redis, Couchbase, HBase



Centralized vs. Distributed vs. Cloud Databases:

Feature	Centralized Database 	Distributed Database 	Cloud Database 
About it	Stores all data in one single server ; all users connect to this main system.	Data is spread across multiple servers that work together as one system.	Data is stored and managed on cloud platforms like AWS, Google Cloud, or Azure.
Data Storage	All data kept in one central location .	Data is divided among several connected nodes .	Data stored in remote data centers managed by providers.
Control & Management	Managed by one organization or admin.	Managed by multiple systems/nodes .	Managed by cloud provider , user controls settings and permissions.
Performance	Fast for local access , but slower for remote users.	High performance through parallel processing and load sharing .	Scalable and fast depending on cloud setup and internet speed.
Scalability	Limited — requires upgrading the central machine.	High — can add more servers easily.	Elastic — scales automatically with demand.
Reliability	Low — failure of the main server stops the system.	High — failure of one node doesn't crash the system.	Very high — automatic backups and redundancy.
Security	Easier — one access point to protect.	Harder — multiple nodes to secure.	Depends on cloud provider security and user configurations .
Cost	Lower setup cost, but higher maintenance as data grows.	Higher setup cost due to infrastructure needs.	Pay-as-you-go , cost-efficient for scaling systems.
Use Cases	 Local systems (schools, offices).	 Large enterprises, banking, telecom.	 Web apps, SaaS platforms, AI-based systems.
Examples	Library database, local hospital system.	Google Spanner, Hadoop HDFS.	Amazon RDS, Google Cloud SQL, Azure SQL DB.



5. Cloud Storage and Databases:

☁ Cloud Storage and Its Relationship to Databases

1. What is Cloud Storage and how does it support databases?

Cloud storage is a service that allows data to be **stored, managed, and accessed over the internet** instead of on a local computer or server 🖥️ ➡️ ☁.

- It provides a **reliable, scalable, and secure environment** for storing data.
- Databases hosted on the cloud (cloud-based databases) use cloud storage to save tables, documents, or key-value data remotely 🌐.
- Cloud storage supports database functionality by enabling:
 - **Automatic backups** 💾
 - **High availability** ⌚
 - **Remote access** from anywhere in the world 🌍

2. Advantages of Using Cloud-Based Databases

- **Scalability** 📈: Easily increase or decrease storage and compute power as needed.
- **High Availability & Reliability** 🛡️: Automatic replication and failover reduce downtime.
- **Cost-Efficiency** 💰: Pay only for what you use (pay-as-you-go model).
- **Maintenance-Free** ⚙️: Cloud providers handle hardware updates, patches, and backups.

- **Global Access** 🌐: Teams can access the database from anywhere.
- **Examples:** Azure SQL, Amazon RDS, Google Cloud Spanner.

3. Disadvantages or Challenges of Cloud-Based Databases

- **Internet Dependency** 🌐: Access requires a stable internet connection; outages can disrupt work.
- **Data Security & Privacy** 🔒: Sensitive data must rely on the provider's security measures.
- **Potential Costs** 💰: Large-scale usage or heavy traffic can become expensive.
- **Limited Customization** ⚙️: Some advanced configurations may not be available compared to on-premises setups.
- **Latency** ⌚: For certain high-speed applications, remote data access can cause delays.